

Phase 2: Vulnerability Correlation & Enrichment

1. Executive Overview & Purpose

Phase 2 (Vulnerability Correlation & Enrichment) serves as the threat intelligence mapping layer within the Automated Penetration Testing Orchestrator pipeline. It bridges the gap between active network reconnaissance and exploit validation.

Taking the structured service banners and version data captured by Phase 1 (scan_result.json), Phase 2 parses product identities, transforms them into standardized Common Platform Enumeration (CPE 2.3) naming syntax, and queries the NIST National Vulnerability Database (NVD) REST API 2.0. The engine retrieves known Common Vulnerabilities and Exposures (CVEs), CVSS v3.1 severity metrics, and technical summaries, exporting enriched vulnerability records to data/vulnerability_results/ for downstream validation.

2. File Directory Breakdown & Component Purposes

The Phase 2 architecture is located within the phase2_correlation/ directory structure.

2.1 phase2_correlation/correlator.py

- **Purpose:** Serves as the central parsing, translation, and vulnerability correlation engine.
- **Core Responsibilities:**
 - **Input Ingestion:** Discovers and ingests Phase 1 scan results from data/scanner_results/, dynamically handling single-target or multi-port output formats.
 - **Service Parsing & Banner Decomposition:** Separates raw software names from version identifiers using robust right-hand delimiter splitting (rsplit(" ", 1)).
 - **CPE 2.3 Translation:** Maps raw service strings to standardized NVD vendor and product identifiers using comprehensive normalization dictionaries (vendor_map and product_map).
 - **NVD REST API 2.0 Integration:** Queries the National Vulnerability Database using the dynamic virtualMatchString parameter to extract CVE records and CVSS base scores.
 - **Rate Limiting & Defensive Retries:** Enforces configurable inter-request delays (time.sleep) to comply with NVD public API throughput thresholds.
 - **Result Serialization:** Exports enriched JSON files containing open ports, service metadata, and associated CVE lists into data/vulnerability_results/.

Phase 2: Vulnerability Correlation & Enrichment

2.2 CLI Integration & Entry Point (v1.2 Update)

- **Purpose:** Provides flexible command-line invocation (`py -3 -m phase2_correlation.correlator`).
- **Core Responsibilities:**
 - Leverages Python's `argparse` to allow operators to specify custom input files via `--file` and custom output destinations via `--output`.
 - Integrates with `config.py` path constants (`SCANNER_RESULTS_DIR` and `VULN_RESULTS_DIR`) as operational defaults.

3. Critical Engineering Points in the Code

- **Flexible `virtualMatchString` vs. Rigid `cpeName` Querying:** The NVD REST API 2.0 parameter `cpeName` strictly demands an exact, fully qualified 13-part CPE entry present in the official NVD dictionary. By migrating to `virtualMatchString` (e.g., `cpe:2.3:a:apache:http_server:2.4.49:*:*:*:*:*`), the correlator allows partial matching and wildcards, successfully capturing CVEs across version ranges without requiring exact dictionary pre-existence.
- **Dynamic Product vs. Version Separation:** Unlike HTTP response headers that delimit software and version by slashes (`Apache/2.4.49`), Nmap service discovery outputs version strings separated by spaces (`Apache httpd 2.4.49`). The correlator implements `version_string.rsplit(" ", 1)` to partition the string from the right, preventing multi-word product names (e.g., `werkzeug httpd`, `apache tomcat`) from having their names prematurely truncated.
- **Enterprise Vendor & Product Translation Dictionaries:** Nmap utilizes descriptive service names, whereas NVD indexes vulnerabilities under official corporate or project umbrellas. The correlator maintains extensive `vendor_map` and `product_map` dictionaries to bridge this taxonomic divide (e.g., translating `nginx` to vendor `f5`, `mysql` to vendor `oracle`, and `werkzeug httpd` to vendor `palletsprojects` and product `werkzeug`).
- **Dynamic Path Resolution & Package Execution:** To prevent `ModuleNotFoundError` across nested project runs, the module dynamically resolves the absolute path via `PROJECT_ROOT = Path(__file__).resolve().parent.parent` and injects it into `sys.path[0]`, allowing seamless import of `config.py` regardless of the caller's working directory.
- **API Rate Limiting Compliance:** To prevent HTTP 403 Forbidden or 429 Too Many Requests errors from the NIST NVD public endpoints during batch multi-port

Phase 2: Vulnerability Correlation & Enrichment

evaluations, the engine implements structured query pacing via configurable `rate_delay` intervals.

4. Problems Encountered and Technical Solutions

Problem Faced	Root Cause	Technical Solution Implemented
Empty NVD Query Results	Raw Nmap service names did not align with NVD corporate umbrellas (e.g., querying NGINX under vendor nginx instead of parent entity f5).	Implemented translation layers (<code>vendor_map</code> and <code>product_map</code>) to systematically map raw service tokens to official NVD organization and product keys.
Strict CPE Query Failures	Initial queries utilized the <code>cpeName</code> parameter, which strictly demands a 13-part exact match in the NVD dictionary.	Transitioned API queries to <code>virtualMatchString</code> , enabling dynamic wildcard matching across software version ranges.
Blank Correlation Output (II)	Upgrading Phase 1 to Nmap changed version delimiters from slashes (/) to spaces (), causing legacy if "/" in <code>version_string</code> conditions to evaluate to False.	Refactored delimiter logic using <code>version_string.rsplit(" ", 1)</code> to cleanly extract version numbers regardless of whether spaces or slashes are used.
API Parameter Accidental Regression	During codebase refactoring, the query function inadvertently reverted from <code>virtualMatchString</code> back to <code>cpeName</code> .	Re-enforced and permanently standardized <code>params = {"virtualMatchString": cpe_name}</code> inside <code>query_nvd_api()</code> .
Nested Directory Import Errors	Running the script from within <code>phase2_correlation/</code> prevented Python from locating the root-level <code>config.py</code> .	Dynamically appended <code>PROJECT_ROOT</code> to <code>sys.path[0]</code> and standardized CLI execution using <code>python -m phase2_correlation.correlator</code> .

Phase 2: Vulnerability Correlation & Enrichment

5. Rationale for Architecture, Tools & Technology Choices

5.1 NIST NVD REST API 2.0 over Local Vulnerability Databases

The NIST NVD is the authoritative, globally recognized repository for CVE records and standardized CVSS v3.1 severity metrics. Directly integrating with the live REST API ensures the orchestrator assesses targets against up-to-date threat data without requiring manual database synchronization or massive local storage overhead.

5.2 Standardized CPE 2.3 Formatting

CPE (Common Platform Enumeration) 2.3 is the standard naming scheme for information technology systems, software, and packages. Translating disparate scanner banners into standardized CPE strings enables unambiguous, reproducible matching across threat intelligence sources.

5.3 Dynamic Vendor/Product Translation Mapping

Different security tools format service banners inconsistently. Building a dedicated translation layer decouples banner discovery from CVE correlation, ensuring that changes to Phase 1 scanner tools do not require rewriting vulnerability query logic.

5.4 Decoupled Data Contracts (JSON File Exchange via data/)

Emitting self-contained JSON records into data/vulnerability_results/ cleanly separates threat intelligence correlation from exploitation testing. This separation of concerns guarantees Phase 3 validation can execute independently or offline without repeatedly querying external APIs.